

ELMAR
CNC-Cutter

Autoren: Alexandra Roth 7025637
David Atrops 7013979
Dennis Smirnow 7023434
Leon Ronge 7026641
Samuel Drees 7026354

Studiengang: Maschinenbau und Design

Erstprüfer: Prof. Dr. Elmar Wings
Malena Wilken

Abgabedatum: 3. Mai 2026

Inhaltsverzeichnis

Inhaltsverzeichnis	3
Abbildungsverzeichnis	5
I. Domain Knowledge - Application	7
II. Domain Knowledge - Hardware	9
1. Sensor: Endschalter	11
1.1. Einleitung: Positionssensoren	11
1.2. Definition und Funktion: Elektromechanischer Endschalter	11
1.3. Specification	11
1.4. Verwendung und Schaltfunktionen	12
1.5. Analyse: Vor- und Nachteile	13
1.6. Ausblick: Intelligente und vernetzte Sensorik	14
1.7. Library	14
1.8. Beispiel	14
1.8.1. Manual	14
1.8.2. Erklärung des Beispiel-Codes	14
1.9. Justage	15
1.10. Tests	15
1.10.1. Einfacher Funktions-Test	15
1.10.2. Manual	16
1.10.3. Einfacher Test-Code	16
1.11. Einfache Anwendung	16
1.11.1. Manual	17
1.12. Weiterführende Literatur	18
2. Schrittmotor	19
2.1. Beschreibung eines Schrittmotors	19
2.1.1. Aufbau und Funktionsweise eines Schrittmotors	19
2.1.2. Schrittmotor Bauformen	20
2.1.3. Betriebsarten unipolar und bipolar	21
2.1.4. Mikroschrittverfahren	21
2.2. Beschreibung des verwendeten Schrittmotors	21
2.3. Beschreibung des Programms auf dem Arduino	22
2.3.1. Aufgabe des Programms	22
2.3.2. Erklärung des Codes	24
2.4. Bibliotheken	27
2.4.1. Accelstepper	27
2.4.2. Wire	30
2.4.3. Adafruit GFX	30
2.4.4. Adafruit SSD1306	30
2.4.5. Encoder	30
2.5. Schrittmotorsteuerung	30

2.6. Weiterführende Literatur	31
III. Domain Knowledge - Tools	33
IV. Developmenmt	35
V. Monitoring	37
VI. Verification - Evaluation - Conclusion	39
VII. Anhang	41
Literaturverzeichnis	43
Literaturverzeichnis	43

Abbildungsverzeichnis

1.1. Darstellung des im Projekt verwendeten Endschalters	12
1.2. Funktionsprinzip eines Mikroschalters mit den Kontakten C, NC und NO	13
2.1. Prinzipieller Aufbau Schrittmotor (2-phasig) [Hag21]	20
2.2. Drehmoment-Diagramm für den Schrittmotor 42BYG015 [Glob] . . .	22
2.3. Flussdiagramm des Programms	23
2.4. Treiberkarte Debo DRV A4988	31

Teil I.

Domain Knowledge - Application

Teil II.

Domain Knowledge - Hardware

1. Sensor: Endschalter

1.1. Einleitung: Positionssensoren

In der industriellen Fertigungssteuerung nimmt die Erfassung von Zuständen eine zentrale Rolle ein. Gemäß der Fachliteratur wandelt ein Sensor (lat. sensus, der Sinn) „eine physikalische Größe (z. B. Kraft oder Temperatur) mit Hilfe eines physikalischen Effekts in ein elektrisches Signal um“ [Her+12, S. 433]. Speziell im Maschinenbau wird bei der Steuerung zwischen zwei grundlegenden Messverfahren unterschieden: Der „Positionserfassung durch Schalter“ und der „Positionserfassung durch Wegbeobachtung“. [Her+12, S. 435]

Bevor der Fokus auf kontaktbehaftete Schalter gelegt wird, sind folgende berührungslose Sensoren (Sensorschalter) als moderner Industriestandard zu nennen:

- Induktive Sensoren: Erfassen metallische Elemente „ohne Kontakt zwischen der Bewegung und dem Sensor“ [Her+12, S. 436].
- Kapazitive Sensoren: Nutzen die Beeinflussung eines elektrischen Feldes zur Kapazitätsänderung [Her+12, S. 438].
- Optische Sensoren: Arbeiten mit Infrarot-Lichtstrahlen zur Objekterkennung (z. B. Lichttaster oder Lichtschranken) [Her+12, S. 440].
- Ultraschall-Sensoren: Basieren auf der Messung der Laufzeit von Schallimpulsen [Her+12, S. 443].

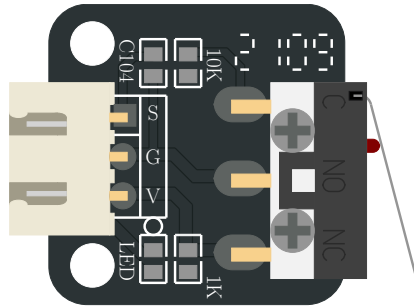
1.2. Definition und Funktion: Elektromechanischer Endschalter

Ein elektromechanischer Endschalter ist ein kontaktbehafteter Positionssensor. Ein solcher Schalter „hat die Aufgabe, das Erreichen einer bestimmten Position (Endlage) zu melden oder einen Schaltvorgang auszulösen“ [Her+12, S. 435]. Dabei wird durch das Betätigen eines mechanischen Elements, wie beispielsweise eines Hebels oder Tasters, ein elektrischer Kontakt geöffnet oder geschlossen [Her+12, S. 120–122].

Technische Spezifikationen (Beispiel Creality): Bei dem vorliegenden Bauteil handelt es sich um einen „originalen Endschalter von Creality 3D, der bei fast allen FDM Druckern von Creality 3D verbaut ist“ [3dj].

1.3. Specification

- Kompatibilität: Geeignet für die X-, Y- oder Z-Achse von Modellen wie der Ender-3 oder CR-10 Serie [3dj][Bas].
- Hersteller: Creality
- Hersteller SKU: 4004180004
- Gewicht: 7.3g
- Abmaße: 28mm x 20mm x 9mm



1.4. Verwendung und Schaltfunktionen

Mechanische Endschalter werden zur Überwachung von Bewegungsabläufen eingesetzt. Dabei können verschiedene logische Funktionen realisiert werden:

- Schließer: Der Kontakt wird bei Betätigung geschlossen.
- Öffner: Der Kontakt wird bei Betätigung unterbrochen.
- Antivalenter Sensor: Dieser verfügt über zwei Ausgänge, wobei „stets ein Kontakt geschlossen und der andere offen“ ist [Her+12, S. 435]. Dies ermöglicht neben der reinen Schaltfunktion auch eine „Fehlererkennung und Diagnose durch die SPS“ [Her+12, S. 435–436].

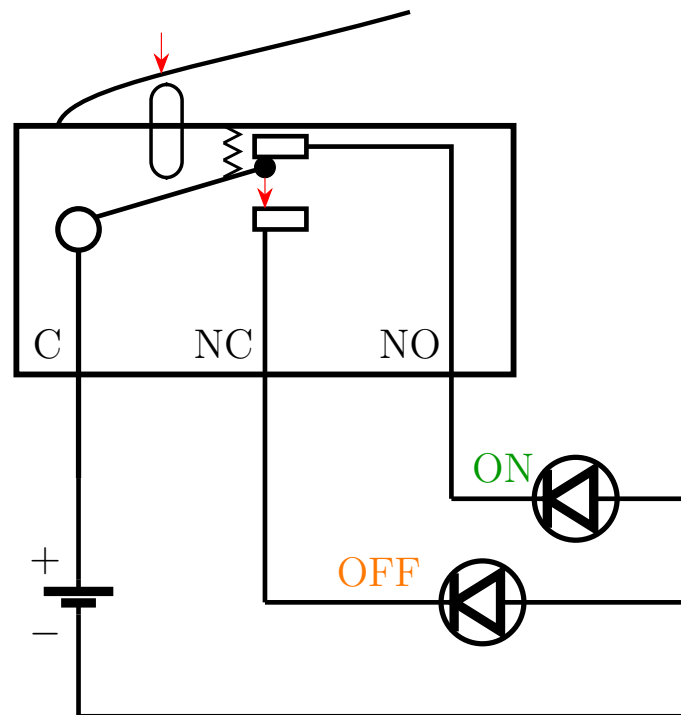


Abbildung 1.2.: Funktionsprinzip eines Mikroschalters mit den Kontakten C, NC und NO

Haupteinsatzgebiete:

- Endstopp-Erkennung: Schutz der Maschine vor dem Überfahren der mechanischen Grenzen.
- Referenzschalter (Homing): Der Schalter dient als „Referenzschalter“, um den Nullpunkt einer Achse (z. B. beim 3D-Drucker) zu kalibrieren [Bas].

1.5. Analyse: Vor- und Nachteile

Vorteile:

- Wirtschaftlichkeit: Sehr kostengünstig (ca. 4-5 € pro Stück) [3DJake, Bastelgarage].
- Einfachheit: Direkte Erzeugung eines binären Signals ohne aufwändige Auswerteelektronik.
- Robustheit gegen EMV: Unempfindlich gegenüber elektromagnetischen Störungen, da kein Oszillator wie bei induktiven Sensoren benötigt wird.

Nachteile:

- Mechanischer Verschleiß: Durch den physischen Kontakt unterliegen die Bauteile einer Abnutzung.
- Geschwindigkeit: Mechanische Kontakte können „prellen“ und sind langsamer als elektronische Lösungen [Her+12, S. 444].
- Technologische Verbreitung: Obwohl immer noch sehr viele mechanische Endschalter eingesetzt werden, ist der berührungslose Endschalter (Sensorschalter) in nahezu allen Bereichen als Standard-Bauelement zu finden [Her+12, S. 435].

1.6. Ausblick: Intelligente und vernetzte Sensorik

Die Zukunft der Positionserfassung liegt in der bidirektionalen Kommunikation. Ein moderner Standard ist hierbei IO-Link, welcher eine „feldbusunabhängige Kommunikation zwischen Sensoren und Aktoren und der Maschinensteuerung“ beschreibt [Her+12, S. 510].

Während klassische Endschalter nur „1 Bit“ an Informationen liefern (Ein/Aus), können intelligente Systeme über IO-Link zusätzlich Servicedaten übertragen, „die zur Einstellung oder Diagnose eines Devices führen“ [Her+12, S. 512].

Dies ermöglicht eine „vorbeugende Instandhaltung“, indem beispielsweise Verschleiß gemeldet wird, bevor es zum Maschinenausfall kommt [Her+12, S. 471, 512].

1.7. Library

Für die Verwendung des Endschalters werden keine zusätzlichen Bibliotheken benötigt. Es werden Funktionen aus der Arduino-Standardbibliothek verwendet:

- `pinMode()`: Konfiguriert einen Pin als Ein- oder Ausgang.
- `digitalRead()`: Liest den aktuellen Zustand eines digitalen Eingangs.
- `digitalWrite()`: Setzt einen digitalen Ausgang auf HIGH oder LOW.
- `Serial.begin()`: Initialisiert die serielle Kommunikation.
- `Serial.print()` / `Serial.println()`: Gibt Daten im Serial Monitor aus.
- `delay()`: Erzeugt eine zeitliche Verzögerung im Programmablauf.

1.8. Beispiel

Dieses Beispiel zeigt, wie der Endschalter über einen digitalen Eingang ausgelesen werden kann. Dabei wird der interne Pull-Up-Widerstand verwendet. Der Zustand des Schalters wird im Serial Monitor angezeigt und ändert sich beim Betätigen von HIGH auf LOW.

1.8.1. Manual

Der Endschalter wird an den digitalen Pin D2 des Mikrocontrollers angeschlossen. Der Signalanschluss des Schalters ist mit D2 verbunden, der zweite Anschluss liegt auf GND.

1.8.2. Erklärung des Beispiel-Codes

Nach der Initialisierung durch die Funktion `setup()` wird das Programm durch die Funktion `loop()` kontinuierlich ausgeführt. Diese Funktion stellt eine Endlosschleife dar, in der der Zustand des Endschalters bei jedem Durchlauf überprüft wird.[Arde; Arde]

Der verwendete Pin wird mit der Funktion `pinMode()` als Eingang oder Ausgang konfiguriert. [Ardf] Dabei wird der Modus `INPUT_PULLUP` verwendet, um den internen Pull-Up-Widerstand zu aktivieren.[Arda] Der interne Pull-Up-Widerstand sorgt dafür, dass am Eingang im Ruhezustand ein definiertes HIGH-Signal anliegt. Innerhalb der `loop()`-Funktion wird der Zustand des Endschalters mit der Funktion `digitalRead()` ausgelesen. Der gelesene Wert wird in einer Variablen gespeichert und über die serielle Schnittstelle ausgegeben.[Ardd] Die Funktionen `Serial.print()` und `Serial.println()` werden verwendet, um den aktuellen Zustand des Eingangs im Serial

Monitor darzustellen.[Ardb] Durch die Verwendung des internen Pull-Up-Widerstands ergibt sich eine invertierte Logik. Im nicht betätigten Zustand des Endschalters liegt ein HIGH-Signal am Eingang an. Wird der Endschalter betätigt liegt ein LOW-Signal an. Zur besseren Lesbarkeit der Ausgabe wird eine Verzögerung mit der Funktion `delay()` erzeugt.[Arde]

Listing 1.1.: Einfaches Beispiel zum Lesen eines mechanischen Endschalters

```

1000 /**
1001  * @file MicroSwitch_BasicRead.ino
1002  * @brief Basic example for reading a micro switch.
1003  */
1004 const int MICRO_SWITCH_PIN = 2; /**< Digital input pin for the micro
1005     switch. */
1006
1007 /**
1008  * @brief Initializes the serial interface and the input pin.
1009  */
1010 void setup()
1011 {
1012     Serial.begin(115200);
1013     pinMode(MICRO_SWITCH_PIN, INPUT_PULLUP);
1014 }
1015
1016 /**
1017  * @brief Reads the switch state and prints it to the serial monitor.
1018  */
1019 void loop()
1020 {
1021     int switchState = digitalRead(MICRO_SWITCH_PIN);
1022
1023     Serial.print("Micro switch state: ");
1024     Serial.println(switchState);
1025
1026     delay(200);
1027 }

```

../Code/MicroSwitchBasicRead/MicroSwitchBasicRead.ino

1.9. Justage

Die Justage (auch "Justierung") ist eine "Tätigkeit, welche ein Messgerät in einen gebrauchstauglichen Betriebszustand versetzt. Sie kann manuell, halb automatisch oder automatisch erfolgen." [Hel, S. 17] Sie kann auch als das "abgleichen des Nullpunktes und der Kennliniensteigung" [Ber24, S. 17] verstanden werden. Da der verwendete Endschalter nur die analogen Werte OPEN = 0 und CLOSED = 1 detektieren kann, sodass keine Kennlinie im herkömmlichen Sinne vorliegt, liegt die anwendungsspezifische Justage-Aufgabe darin, den gesamten Endschalter physisch in seiner Position am Rahmengestell so auszurichten, dass er die entsprechende bewegliche Achse im Rahmen einer Referenzfahrt in ihrer Bewegung eingrenzt, aber gleichzeitig im Normalbetrieb den erforderlichen Bauraum nicht künstlich einschränkt.

1.10. Tests

1.10.1. Einfacher Funktions-Test

Im einfachsten Anwendungstest wird der Endschalter verwendet, um eine LED zu steuern. Hierbei wird der Zustand des Schalters ausgewertet und auf einen Ausgang übertragen. Wird der Endschalter betätigt, schaltet sich die LED ein. Im nicht betä-

tigten Zustand bleibt die LED ausgeschaltet. Durch diesen Test kann die Funktion des Sensors sowie die Verarbeitung des Signals im System überprüft werden.

1.10.2. Manual

Der Endschalter wird an den digitalen Pin D2 des Mikrocontrollers angeschlossen. Der Signalanschluss des Schalters ist mit D2 verbunden, der zweite Anschluss liegt auf GND. Die LED wird an einen digitalen Ausgang (z. B. D13) angeschlossen.

Beim Betätigen des Endschalters wird die LED eingeschaltet, im nicht betätigten Zustand bleibt sie ausgeschaltet.

1.10.3. Einfacher Test-Code

Die Funktion `loop()` läuft dauerhaft und liest den Zustand des Mikroschalters ein. Da `INPUT_PULLUP` verwendet wird, bedeutet `LOW` = gedrückt. Ist der Schalter gedrückt, wird die LED eingeschaltet, sonst ausgeschaltet.

Listing 1.2.: An- und Ausschalten einer LED mittels Endschalter

```

1000 /**
1001  * @file MicroSwitch_LEDExample.ino
1002  * @brief Simple application example using a micro switch and an LED.
1003  */
1004
1005 const int MICRO_SWITCH_PIN = 2; /**< Digital input pin for the micro
1006     switch. */
1007 const int LED_PIN = 13;          /**< Digital output pin for the built-in
1008     LED. */
1009
1010 /**
1011  * @brief Initializes the input and output pins.
1012  */
1013 void setup()
1014 {
1015     pinMode(MICRO_SWITCH_PIN, INPUT_PULLUP);
1016     pinMode(LED_PIN, OUTPUT);
1017 }
1018
1019 /**
1020  * @brief Switches the LED depending on the micro switch state.
1021  */
1022 void loop()
1023 {
1024     int switchState = digitalRead(MICRO_SWITCH_PIN);
1025
1026     if (switchState == LOW)
1027     {
1028         digitalWrite(LED_PIN, HIGH);
1029     }
1030     else
1031     {
1032         digitalWrite(LED_PIN, LOW);
1033     }
1034 }

```

../Code/MicroSwitchLEDExample/MicroSwitchLEDExample.ino

1.11. Einfache Anwendung

Das Programm führt eine Referenzfahrt mit einem Schrittmotor aus.

Im `setup()` werden die Pins für den Treiber und den Endschalter eingerichtet. Danach wird der Motortreiber aktiviert und die Referenzfahrt gestartet.

Die Funktion `oneStep()` erzeugt einen einzelnen Schritt des Motors.

Mit `moveSteps()` kann der Motor eine festgelegte Anzahl an Schritten in eine bestimmte Richtung fahren.

In `homeAxis()` fährt der Motor so lange, bis der Endschalter gedrückt wird. Danach wartet das Programm kurz und fährt den Motor einige Schritte zurück.

1.11.1. Manual

Der Endschalter wird an den digitalen Pin D2 des Mikrocontrollers angeschlossen. Der Signalanschluss des Schalters wird an D2 angeschlossen, der zweite Anschluss liegt auf GND. Der Schrittmotortreiber wird über die Steuerpins STEP und DIR an digitale Ausgänge des Mikrocontrollers angeschlossen (z. B. STEP an D3 und DIR an D4).

Listing 1.3.: Referenzfahrt einer Achse mit Endschalter

```

1000 /**
      * @file referenzfahrt.ino
1002  * @brief Homing routine using a stepper motor, A4988 driver, and
      *       endstop.
      */
1004
1005 /** @brief STEP pin of the driver */
1006 const int stepPin = 2;
1007
1008 /** @brief Direction pin of the driver */
1009 const int dirPin = 5;
1010
1011 /** @brief Enable pin of the driver */
1012 const int enPin = 8;
1013
1014 /** @brief Endstop input pin */
1015 const int endstopPin = 9;
1016
1017 /** @brief Step delay in microseconds */
1018 const int stepDelayUs = 1000;
1019
1020 /** @brief Number of steps for the backoff movement */
1021 const int backoffSteps = 50;
1022
1023 /** @brief Initializes the hardware and starts the homing routine */
1024 void setup() {
1025     pinMode(stepPin, OUTPUT);
1026     pinMode(dirPin, OUTPUT);
1027     pinMode(enPin, OUTPUT);
1028     pinMode(endstopPin, INPUT_PULLUP);
1029
1030     Serial.begin(115200);
1031
1032     digitalWrite(enPin, LOW);
1033     digitalWrite(dirPin, LOW);
1034
1035     homeAxis();
1036 }
1037
1038 /** @brief Main loop (unused in this example) */
1039 void loop() {
1040 }
1041
1042 /** @brief Executes a single motor step */
1043 void oneStep() {
1044     digitalWrite(stepPin, HIGH);
1045     delayMicroseconds(stepDelayUs);
1046     digitalWrite(stepPin, LOW);

```

```
    delayMicroseconds(stepDelayUs);
1048 }

1050 /**
1051  * @brief Moves the motor by a defined number of steps
1052  * @param steps Number of steps to execute
1053  * @param dir Direction of movement (LOW/HIGH)
1054  */
1055 void moveSteps(int steps, bool dir) {
1056     digitalWrite(dirPin, dir);
1057     for (int i = 0; i < steps; i++) {
1058         oneStep();
1059     }
1060 }

1062 /** @brief Performs the homing sequence and backoff movement */
1063 void homeAxis() {
1064     while (digitalRead(endstopPin) == HIGH) {
1065         oneStep();
1066     }

1068     delay(200);
1069     moveSteps(backoffSteps, HIGH);
1070 }
```

../Code/referenzfahrt/referenzfahrt.ino

1.12. Weiterführende Literatur

- Zielke, Dirk: *Mikrosysteme*. Springer, 2025. [Zie25] Das Buch behandelt die Grundlagen und Anwendungen von Mikrosystemen und bietet einen Überblick über Aufbau, Funktion und Einsatz moderner Sensorsysteme.
- Tränkler, Hans-Rolf und Reindl, Leo: *Sensortechnik*. Springer, 2014. [TR14] Das Buch behandelt grundlegende Prinzipien der Sensortechnik und beschreibt verschiedene Sensorarten sowie deren Einsatz in technischen Systemen.

2. Schrittmotor

In diesem Kapitel folgt eine grundsätzliche Beschreibung eines Schrittmotors. Darauf folgt die Erläuterungen zum Aufbau und Funktionsweise eines Schrittmotors, sowie die Aufzählung verschiedener Grundbauarten. Nachfolgend werden Ursachen und Lösungen zum Verhindern von Schrittfehlern und der verwendete Schrittmotor genauer erläutert.

2.1. Beschreibung eines Schrittmotors

Ein Schrittmotor ist ein Elektromotor, der sich für präzise Positionierungsaufgaben eignet. Im Gegenteil zu anderen Elektromotoren wird bei einem Schrittmotor keine Positionsmessung oder Positionsregelung benötigt. Andere mechanische Antriebssysteme benötigen einen geschlossenen Regelkreis und mechanische Bremsen, um Drehzahl und Position einzuhalten. Der Motor führt durch die Rotation des Rotors diskrete Schritte aus, wobei jeder Steuerimpuls eine Verschiebung um einen konstanten Winkel bewirkt. Diese Art von Elektromotor wird beispielsweise in Druckern oder Scannern, aber auch im Kraftfahrzeugbereich verwendet. Im Kraftfahrzeugbereich werden die Schrittmotoren zur Spiegelverstellung sowie der Sitzverstellung verwendet. Der Vorteil eines Schrittmotors ist die Wartungsfreiheit, da der Rotor keine Wicklungen hat. [Bab23; Hag21; Ber18; SK21]

2.1.1. Aufbau und Funktionsweise eines Schrittmotors

Der Schrittmotor besteht aus einem Stator, einem Rotor ohne Wicklungen und einer Steuerelektronik. Diese Steuerelektronik setzt sich zusammen aus einer Treiberstufe und der eigentlichen Steuerung. Bei dem Stator handelt es sich um den feststehenden äußeren Teil und bei dem Rotor um den beweglichen inneren Teil, wie in Abbildung 2.1 zu erkennen ist. Im Stator sind Spulen verbaut, die von einem Strom durchflossen werden. Hierdurch entsteht ein magnetisches Feld. Da der Rotor magnetisch ist, folgt er dem Magnetfeld des Stators. Soll eine Bewegung hervorgerufen werden, werden einzelne Wicklungsstränge ein-, aus- oder umgeschaltet. Durch diesen Vorgang wird ein rotierendes Magnetfeld erzeugt. Das erzeugte Magnetfeld zieht den Rotor an. Der Rotor wird bei jedem Puls (Takt) um einen Winkelschritt weitergeschaltet. Die Anzahl der Polpaare im Stator gibt die Anzahl der Schritte vor. Die Drehzahl und die Drehrichtung hängen von der Reihenfolge und der Häufigkeit der Stromimpulse ab. Es gibt drei verschiedene Betriebsarten, die abhängig von der Genauigkeit und der Drehzahl sind. Im Vollschrittbetrieb werden alle Polpaare bestromt. Im Halbschrittbetrieb wird die Schrittzahl des Motors verdoppelt, wodurch sich die Positionsauflösung im Gegensatz zum Vollschrittbetrieb verdoppelt. Allerdings wird in diesem Betrieb das Drehmoment reduziert. Im Mikroschrittbetrieb bewegt sich der Rotor in nochmals kleineren Schritten. Hierdurch werden eine hohe Positioniergenauigkeit und ein ruhiger Lauf erreicht, da die Ströme und das Drehmoment in kleineren Schritten verändert werden. Bei einer zu hohen Schrittfrequenz kann ein Schrittverlust entstehen. Bei einem Schrittverlust überspringt der Motor einzelne Winkelschritte und landet in einer vorherigen oder nächsten Position gleicher Phase. Durch die offene Steuerkette werden die Verluste nicht erkannt. [Hag21; Ber18; SK21]

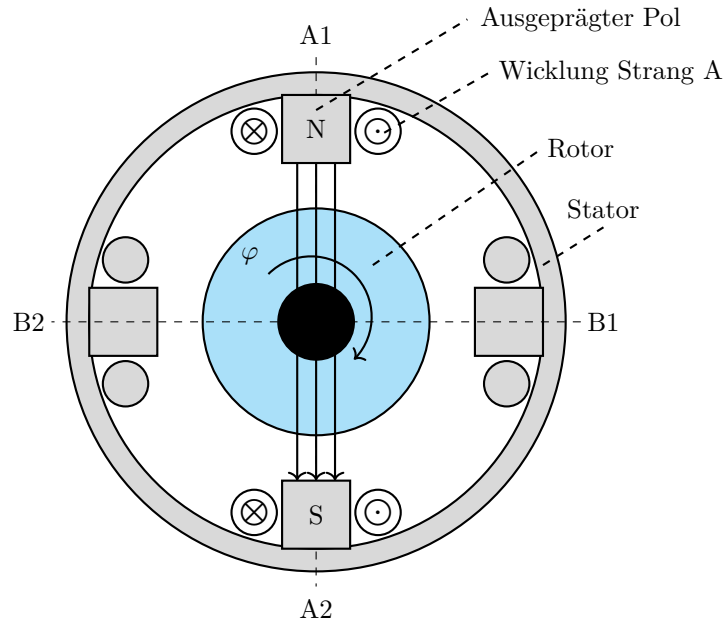


Abbildung 2.1.: Prinzipieller Aufbau Schrittmotor (2-phasig) [Hag21]

2.1.2. Schrittmotor Bauformen

Wie bei anderen Elektromotoren gibt es auch bei dem Schrittmotor verschiedene Grundbauarten.

- Permanentmagnetereger-Schrittmotor (PM-Schrittmotor)
- Reluktanzschrittmotor (VR-Schrittmotor)
- Hybridschrittmotor (Hybrid-Schrittmotor)

Der **permanentmagnetische Schrittmotor** hat einen Permanentmagneten in dem Rotor verbaut. Hierbei stellt sich der permanentmagnetische Rotor immer so, dass der Nordpol des Rotors dem Nordpol des Statorfeldes gegenüber liegt und der Südpol des Rotors dem Südpol des Statorfeldes. In dieser Ausrichtung ziehen sich die Pole gegenseitig an. Die Drehrichtung des Rotors hängt von der Fließrichtung des Stromes ab. Der permanentmagnetische Schrittmotor entwickelt im ausgeschalteten Zustand ein Drehmoment zur Selbsthaltung. Dies ist aufgrund des permanentmagnetischen Rotors möglich. Bei diesem Drehmoment handelt es sich um das höchste Drehmoment, das auf die Welle des Motors übertragen werden kann, ohne dass diese sich in eine rotierende Bewegung versetzt. Zu dieser Art von Schrittmotoren gehören beispielsweise der Klauenpol-Schrittmotor und der Scheibenmagnet-Schrittmotor. Bei der zweiten Bauart handelt es sich um den **Reluktanzschrittmotor**. Bei dieser Bauart besteht der Rotor aus einem weichmagnetischen Material und besitzt eine gezahnte Form. So lange der Schrittmotor von keinem Strom durchflossen wird, entsteht kein Magnetfeld. Sobald der Motor in Betrieb genommen wird, entsteht ein magnetischer Fluss innerhalb des Rotors. Wird nun eine Wicklung erregt, wird der nächste Zahn des Rotors angezogen. Dadurch, dass die Rotorzähne ungleich der Polteilung sind, kann das System unendlich lange fortgesetzt werden. Die Anzahl der Schritte und die Genauigkeit des Reluktanzschrittmotors ist abhängig von der Anzahl der Zähne auf dem Rotor. Aus technischer Sicht sind mit dieser Bauart Schrittwinkel unter 1° möglich. Damit die Drehrichtung verändert werden kann, sind mindestens zwei Strangwicklungen nötig. Bei den **Hybridschrittmotoren** handelt es sich um eine Kombination aus

dem Reluktanzschrittmotor und dem Permanentmagneterreger-Schrittmotor. Durch diese Kombination aus den beiden Schrittmotoren werden die Vorteile aus der kleinen Schrittweite, dem hohen Drehmoment und dem Selbsthaltungsmoment genutzt. Bei dem Hybridschrittmotor besteht der Rotor aus zwei um eine halbe Zahnteilung versetzten weichmagnetischen Polrädern, die eine zahnförmige Form haben. Bei den beiden Polrädern bildet das eine Polrad den Nordpol und das zweite Polrad den Südpol. Zwischen den beiden Polrädern befindet sich ein Permanentmagnet. Anders als bei anderen Schrittmotoren wird der Rotor bei dieser Bauart axial magnetisiert. Damit ein kleiner Schrittwinkel möglich ist, haben die Statorpole ebenfalls eine zahnförmige Form. Die Ausrichtung des Rotors ist abhängig von der Stromrichtung und wird durch den minimalen Widerstand bestimmt, der sich aus dem Stromfluss durch die einzelnen Stränge ergibt. Wird ein besonders kleiner Schrittwinkel benötigt, kann dies durch Erhöhung der Zähnezahzahl erreicht werden. [SK21; Hag21; Bab23]

2.1.3. Betriebsarten unipolar und bipolar

Neben den oben bereits genannten Betriebsarten, kann außerdem zwischen dem Unipolarbetrieb und dem Bipolarbetrieb unterschieden werden. Der große Unterschied zwischen den beiden Betriebsarten besteht darin, dass in dem Unipolarbetrieb der Strom in eine Richtung fließt. Bei dem Bipolarbetrieb hingegen fließt der Strom in beide Richtungen. Dies ist möglich, da jeder Wicklungsstrang über eine Vollbrücke gespeist wird. Ein weiterer Unterschied besteht in der Schaltung der Zweige, durch die ein Gleichstrom fließt. In dem Unipolarbetrieb werden die beiden Zweige in Reihe geschaltet. Jeder Wicklungsstrang wird mit zwei Drähten parallel gewickelt. Sind die beiden Wicklungsstränge in dem Bipolarbetrieb parallel gewickelt, müssen die Zweige parallel geschaltet werden. Im Bipolarbetrieb kann ein höherer Wirkungsgrad erzielt werden, wo hingegen der Unipolarbetrieb eine deutlich einfachere Schaltung aufweist. [SK21]

2.1.4. Mikroschrittverfahren

Mit dem Mikroschrittverfahren können Zwischenschritte und Mikroschritte erzeugt werden. Es bietet die Fähigkeit, Geräusche und Vibrationen zu minimieren sowie die Auflösung der Umdrehung zu erhöhen. Um Zwischenschritte zu erreichen, müssen beide Wicklungen eines Schrittmotors gleichzeitig mit Strom versorgt werden. Mit dem Sinus/Cosinus-Mikroschrittverfahren können die Mikroschrittverfahren realisiert werden, indem der Strom in jeder Phase des Motors sinusförmig variiert. Durch die Anwendung sinusförmiger Ströme auf die Motorwicklungen wird eine feinere und gleichmäßigere Bewegung erzielt, was zur einer präziseren Steuerung und reduzierten mechanischen Vibrationen führt. [McC24c]

Ein Mikroschritt-Treiber unterteilt die Motorschrittwinkel in vier oder mehr Teile. Es handelt sich um einen Controller, der die Methode der Stromreglung verwendet, um die Mikroschritte zu erzeugen. Der Controller bietet außerdem den Vorteil einer erhöhten Flexibilität bei der Integration der Strombegrenzung und der Anpassung der Antriebscharakteristik als Reaktion auf Änderungen der Systemdynamik. Die Anzahl der Unterteilungen ist einstellbar, was eine präzise Steuerung und Anpassung an spezifische Anforderungen ermöglicht. [Bab23]

2.2. Beschreibung des verwendeten Schrittmotors

Für das Automatisierungsprojekt wird ein Nema 17 Schrittmotor der Firma Creality3D verwendet. Dieser Schrittmotor wird im 3D-Druck sowie in CNC-Maschinen eingesetzt. Der Motor arbeitet im Bipolarbetrieb und es sind zwei Spulen verbaut. Er hat einen Schrittwinkel von $1,8^\circ$ und benötigt 200 Schritte für eine volle Umdrehung. Die

Wellenlänge beträgt 20 mm und der Wellendurchmesser 5 mm. Der ausgewählte Schrittmotor arbeitet mit einer Nennspannung von 12 Volt. Der Motor ist nicht für große Drehmomente ausgelegt, sondern für schnelle und präzise Bewegungen. Dies wird qualitativ in der Abbildung 2.2 für den Schrittmotor GM42BYG015 dargestellt. Hier ist zu erkennen, dass der Schrittmotor ein Drehmoment von ca. 0,8 kg·cm erreicht. Auf der x-Achse wird die Geschwindigkeit in Pulses per Second (PPS) angegeben, d.h. die Anzahl der Steuerimpulse pro Sekunde. Der Schrittmotor erzielt aber eine gute Positioniergenauigkeit durch den kleinen Schrittwinkel und die Microstepp-Funktion. Des weiteren arbeitet der Motor mit 0,84 Amper pro Phase. Weitere Vorteile des Schrittmotors sind die kompakte Bauform mit den Abmessung $42 \times 42 \times 34$ mm sowie die geringe Masse mit 220 Gramm. Betrieben werden kann der Schrittmotor in einer Umgebungstemperatur von $-10\text{ }^{\circ}\text{C}$ bis $+50\text{ }^{\circ}\text{C}$. [Glob; Gloa]

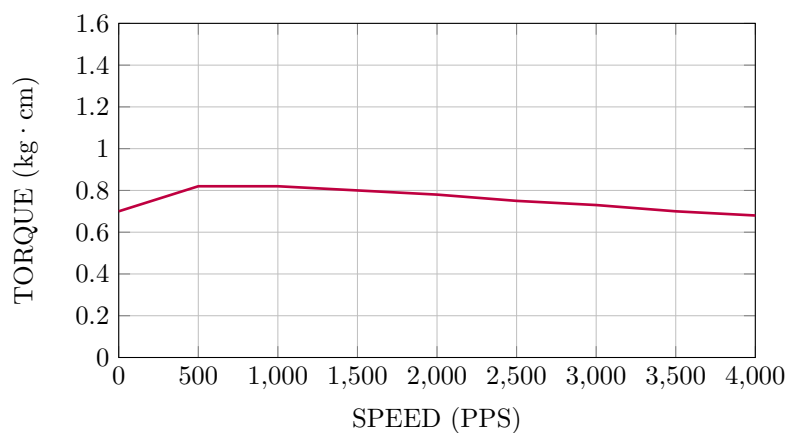


Abbildung 2.2.: Drehmoment-Diagramm für den Schrittmotor 42BYG015 [Glob]

2.3. Beschreibung des Programms auf dem Arduino

2.3.1. Aufgabe des Programms

Das Programm steuert einen Schrittmotor und zeigt Informationen auf einem OLED-Display an. Es nutzt einen Drehwinkel-Encoder, um verschiedene Betriebsstufen des Motors auszuwählen.

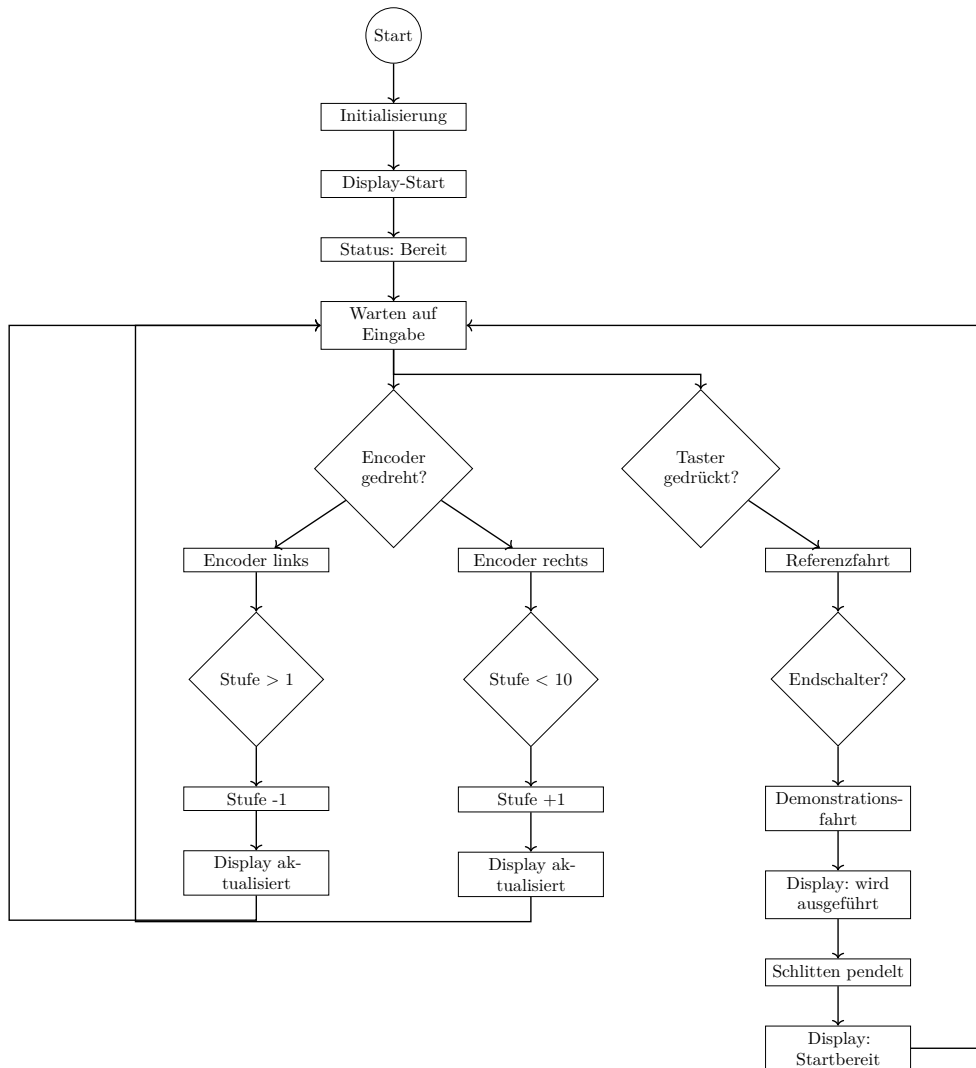


Abbildung 2.3.: Flussdiagramm des Programms

Beim Start des Programms werden alle Komponenten, einschließlich des Schrittmotors, des Displays, der LEDs und des Encoders, initialisiert. Nach einer kurzen Startanimation auf dem Display wechselt das System in den Bereitschaftsmodus. Der Benutzer kann den Drehwinkel-Encoder drehen, um eine der zehn verfügbaren Stufen auszuwählen. Jede Stufe hat spezifische Geschwindigkeitswerte für den Schrittmotor. Die aktuell ausgewählte Stufe wird auf dem OLED-Display angezeigt, sodass der Benutzer immer über die gewählte Einstellung informiert ist. Ein Taster ermöglicht es dem Benutzer, den Startprozess des Schrittmotors zu initiieren. Nach dem Drücken des Tasters führt der Schrittmotor Bewegungen aus, die der ausgewählten Stufe entsprechen. Diese Bewegungen beinhalten das Fahren zu einem festgelegten Punkt und das anschließende Zurückfahren zur Ausgangsposition. Zusammengefasst ermöglicht das Programm dem Benutzer, durch Drehen des Encoders verschiedene Betriebsstufen auszuwählen und den Schrittmotor zu steuern. Die Stufenanzeige auf dem Display und die LED-Statusanzeigen bieten dabei eine klare und leicht verständliche Rückmeldung über den aktuellen Zustand des Systems. Der Ablauf des Programms ist in Abbildung 2.3 dargestellt.

```

1      #include "AccelStepper.h"
2      #include <Wire.h>
3      #include <Adafruit_GFX.h>
4      #include <Adafruit_SSD1306.h>
5      #include <Encoder.h>

```

Listing 2.1.: Einbindung der Bibliotheken

```

1      #define MOTOR_INTERFACE_TYPE 1
2      #define SCREEN_WIDTH 128
3      #define SCREEN_HEIGHT 64
4      #define OLED_RESET -1
5      #define SCREEN_ADDRESS 0x3C
6
7      #define PIN_STEP_DIR 4
8      #define PIN_STEP_STEP 5
9      #define PIN_LED_RED 6
10     #define PIN_LED_GREEN 3
11     #define PIN_LED_BLUE 9
12     #define PIN_BUTTON 11
13     #define PIN_ENDSTOP 12
14     #define PIN_CLK 9
15     #define PIN_DT 2
16     #define PIN_SW 3

```

Listing 2.2.: define-Anweisungen

2.3.2. Erklärung des Codes

Das Programm zur Steuerung des Demonstrators wurde zunächst aus Teilen der Testprogramme für jede Hardware-Komponente basierend auf der gewünschten Funktion zusammengefügt. Nachdem die gewünschte Funktionalität erreicht war, wurde der Code mit Hilfe des Large-Language-Models Chat GPT in der Version 4.0 des US-Unternehmens OpenAI Inc. bezüglich der Lesbarkeit und Vereinheitlichung Benennung von Variablen. Zuerst werden die notwendigen Bibliotheken eingebunden, die für die Steuerung des Schrittmotors, das Display und den Encoder erforderlich sind. 2.1

Die define-Anweisungen legen Konstanten fest, wie die Art des Motorinterfaces, die Abmessungen des Displays und die Pin-Nummern für den Stepper-Motor, die LEDs, den Taster, den Endschalter und den Encoder. 2.2

Nun werden die Objekte für den Stepper-Motor, das Display und den Encoder erstellt.

```

1      AccelStepper stepper(MOTOR_INTERFACE_TYPE, PIN_STEP_STEP,
2                          PIN_STEP_DIR);
3      Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire,
                              OLED_RESET);
4      Encoder encoder(PIN_DT, PIN_CLK);

```

Die globalen Variablen speichern den Status des Tasters, die Zeit der letzten Zustandsänderung, die aktuelle Stufe sowie die Geschwindigkeiten und Beschleunigungen für jede Stufe. 2.3

In der setup-Funktion werden die Startwerte für die maximale Geschwindigkeit und Beschleunigung des Schrittmotors gesetzt, die Pins für LEDs, Taster und Endschalter initialisiert, die serielle Kommunikation gestartet und das Display initialisiert. Ein Interrupt für den Encoder-Taster wird festgelegt. Das Display zeigt eine Startanimation, und die LEDs werden auf ihre Startzustände gesetzt. 2.4


```

1      int buttonStatus = HIGH;
2      int lastButtonStatus = HIGH;
3      unsigned long lastDebounceTime = 0;
4      unsigned long debounceDelay = 50;
5      int endstopStatus = HIGH;
6      int lastEndstopStatus = HIGH;
7      unsigned long lastEndstopDebounceTime = 0;
8      int stage = 0;
9      int speeds[] = {1000,
10                     500,
11                     750,
12                     1000,
13                     1250,
14                     1500,
15                     1750,
16                     2000,
17                     2250,
18                     2500,
19                     1750};
20      int accelerations[] = {1000,
21                             6000,
22                             6000,
23                             6000,
24                             6000,
25                             6000,
26                             6000,
27                             6000,
28                             6000,
29                             6000,
30                             6000};
31      long oldPosition = -999;
32      const int safeSteps = 70;
33      const int trackLength = 1750;
34      const char* stageMessages[] = {"Keine_Stufe_eingestellt",
35                                     "Stufe_1",
36                                     "Stufe_2",
37                                     "Stufe_3",
38                                     "Stufe_4",
39                                     "Stufe_5",
40                                     "Stufe_6",
41                                     "Stufe_7",
42                                     "Stufe_8",
43                                     "Stufe_9",
44                                     "Stufe_10"};

```

```

1      const char* messages[] = {"Start...",
2                                "Startbereit",
3                                "Programm_wird_ausgefuehrt...",
4                                "Vorgang_abgebrochen,_das_Geraet
5                                muss_neu_gestartet_werden",
6                                ""};
7

```

Listing 2.3.: globale Variablen

```
1      void setup() {
2          stepper.setMaxSpeed(1000);
3          stepper.setAcceleration(1000);
4          pinMode(PIN_LED_RED, OUTPUT);
5          pinMode(PIN_LED_GREEN, OUTPUT);
6          pinMode(PIN_LED_BLUE, OUTPUT);
7          pinMode(PIN_BUTTON, INPUT_PULLUP);
8          pinMode(PIN_ENDSTOP, INPUT_PULLUP);
9
10         Serial.begin(9600);
11         if (!display.begin(SSD1306_SWITCHCAPVCC,
12                             SCREEN_ADDRESS)) {
13             Serial.println(F("SSD1306_allocation_failed"));
14             for (;;);
15         }
16
17         attachInterrupt(digitalPinToInterrupt(PIN_SW),
18                         handleInterrupt, CHANGE);
19
20         display.display();
21         delay(2000);
22         display.setTextSize(2);
23         display.setTextColor(SSD1306_WHITE);
24         updateDisplay(messages[0], messages[4]);
25         delay(1000);
26         updateLEDs(false, true, false);
27     }
```

Listing 2.4.: void setup

Die loop-Funktion wird kontinuierlich ausgeführt. Sie liest die Position des Encoders aus und zeigt bei einer Änderung die entsprechende Stufe auf dem Display an. Der Zustand des Tasters wird abgefragt und bei Betätigung die Referenzfahrt des Motors gestartet. Nach der Referenzfahrt führt der Motor Bewegungen gemäß den eingestellten Stufen und Geschwindigkeiten aus. 2.5

Die updateLEDs-Funktion aktualisiert den Zustand der LEDs basierend auf den übergebenen Parametern. 2.6

Die updateDisplay-Funktion aktualisiert das Display mit den übergebenen Nachrichten. 2.7

Die findReference-Funktion führt die Referenzfahrt aus und überprüft den Endschalter. Wenn der Endschalter gedrückt ist, stoppt der Motor und die Funktion gibt true zurück. 2.8

Die handleInterrupt-Funktion wird aufgerufen, wenn der Encoder-Taster gedrückt wird. Sie stoppt den Motor und aktualisiert das Display mit einer Abbruchnachricht. 2.9

2.4. Bibliotheken

Für die Erstellung des Programms werden Bibliotheken verwendet, um die Programmierung zu vereinfachen.

2.4.1. Accelstepper

Die Arduino AccelStepper Bibliothek bietet eine objektorientierte Schnittstelle für die Steuerung von Schrittmotoren und Motortreibern über 2, 3 oder 4 Pins. Diese Bibliothek stellt eine wesentliche Verbesserung gegenüber der standardmäßigen Arduino Stepper Bibliothek dar und bietet zahlreiche erweiterte Funktionen, die für anspruchsvollere Anwendungen erforderlich sind. Sie wird in diesem Projekt in der Version 1.64 verwendet. Zu den herausragenden Merkmalen der AccelStepper Bibliothek gehört die Unterstützung von Beschleunigungs- und Verzögerungsprozessen. Dies ermöglicht eine feinere Steuerung der Motorbewegungen, was insbesondere bei präzisen Anwendungen von Vorteil ist. Darüber hinaus unterstützt die Bibliothek die gleichzeitige Steuerung mehrerer Schrittmotoren, wobei jeder Motor unabhängig voneinander gesteuert werden kann. Dies ist besonders nützlich in Anwendungen, die eine synchronisierte Bewegung mehrerer Motoren erfordern. Ein weiterer Vorteil der AccelStepper Bibliothek ist, dass die meisten API-Funktionen nicht blockieren oder verzögern, es sei denn, dies ist ausdrücklich angegeben. Dies trägt dazu bei, dass der Hauptprogrammfluss nicht unterbrochen wird, was die Effizienz und Reaktionsfähigkeit des Systems verbessert. Die Bibliothek unterstützt sowohl 2-, 3- als auch 4-Draht-Schrittmotoren sowie 3- und 4-Draht-Halbschrittmotoren. Darüber hinaus können alternative Schrittverfahren verwendet werden, um beispielsweise den AFMotor zu unterstützen. Die Theorie hinter der AccelStepper Bibliothek basiert auf den Geschwindigkeitsberechnungen, die in dem Artikel „Generate stepper-motor speed profiles in real time“ von David Austin beschrieben sind. Die Bibliothek verwendet Schritte pro Sekunde, anstelle von Radianen pro Sekunde, da der Schrittwinkel des Motors nicht bekannt ist. Ein anfängliches Schrittintervall wird basierend auf der gewünschten Beschleunigung berechnet, und bei nachfolgenden Schritten werden kürzere Schrittintervalle basierend auf dem vorherigen Schritt berechnet, bis die maximale Geschwindigkeit erreicht ist. Die AccelStepper Bibliothek wird unter einer freien GPL-Lizenz angeboten, was bedeutet, dass sie kostenlos genutzt und verändert werden kann, solange der Quellcode offengelegt wird. Für kommerzielle Anwendungen, bei denen der Quellcode nicht offengelegt werden soll, steht eine kommerzielle Lizenz zur Verfügung. Zusammenfassend bietet die Arduino AccelStepper Bibliothek eine leistungsstarke und flexible Lösung für die Steuerung von Schrittmotoren. Mit ihrer Unterstützung für Beschleunigung und Verzögerung,

```

1      void loop() {
2          long newPosition = encoder.read();
3          if (newPosition != oldPosition) {
4              stage = constrain(newPosition / 4, 1, 10);
5              updateDisplay(messages[1], stageMessages[
6                  stage]);
7              oldPosition = newPosition;
8          }
9
10         int buttonReading = digitalRead(PIN_BUTTON);
11         if (buttonReading != lastButtonStatus) {
12             lastDebounceTime = millis();
13         }
14
15         if ((millis() - lastDebounceTime) > debounceDelay) {
16             if (buttonReading != buttonStatus) {
17                 buttonStatus = buttonReading;
18                 if (buttonStatus == LOW) {
19                     stepper.setAcceleration(
20                         accelerations[0]);
21                     while (!findReference()) {
22                         stepper.setSpeed
23                             (-200);
24                         stepper.runSpeed();
25                     }
26
27                     int currentStage = stage;
28                     updateDisplay(messages[2],
29                         stageMessages[
30                             currentStage]);
31                     updateLEDs(false, false,
32                         true);
33                     for (int i = 0; i < 2; i++)
34                     {
35                         if (i == 0) {
36                             stepper.
37                                 setMaxSpeed
38                                 (speeds
39                                 [
40                                     currentStage
41                                 ]);
42                             stepper.
43                                 setAcceleration
44                                 (
45                                     accelerations
46                                     [
47                                         currentStage
48                                     ]);
49                             stepper.move
50                                 (
51                                     trackLength
52                                     +
53                                     safeSteps
54                                 );
55                             while (
56                                 stepper
57                                 .
58                                 distanceToGo
59                                 () !=
60                                 0) {
61                                 stepper
62                                    .
63                                    run
64                                    ()
65                                    ;
66                             }
67                         }
68                     }
69                 }
70             }
71         }
72     }
73 }

```

```
1 void updateLEDs(bool red, bool green, bool blue) {  
2     digitalWrite(PIN_LED_RED, red ? HIGH : LOW);  
3     digitalWrite(PIN_LED_GREEN, green ? HIGH : LOW);  
4     digitalWrite(PIN_LED_BLUE, blue ? HIGH : LOW);  
5 }
```

Listing 2.6.: void updateLEDs

```
1 void updateDisplay(const char* line1, const char* line2) {  
2     display.clearDisplay();  
3     display.setCursor(0, 0);  
4     display.println(F(line1));  
5     if (line2 != "") {  
6         display.println(F(line2));  
7     }  
8     display.display();  
9 }
```

Listing 2.7.: void updateDisplay

```
1 bool findReference() {  
2     int endstopReading = digitalRead(PIN_ENDSTOP);  
3     if (endstopReading != lastEndstopStatus) {  
4         lastEndstopDebounceTime = millis();  
5     }  
6  
7     if ((millis() - lastEndstopDebounceTime) >  
8         debounceDelay) {  
9         if (endstopReading != endstopStatus) {  
10             endstopStatus = endstopReading;  
11             if (endstopStatus == LOW) {  
12                 stepper.stop();  
13                 return true;  
14             }  
15         }  
16         lastEndstopStatus = endstopReading;  
17         return false;  
18     }
```

Listing 2.8.: bool findReference

```
1 void handleInterrupt() {  
2     stepper.stop();  
3     updateDisplay(messages[3], messages[4]);  
4 }
```

Listing 2.9.: void handleInterrupt

der gleichzeitigen Steuerung mehrerer Motoren und ihrer nicht blockierenden API ist sie eine ausgezeichnete Wahl für anspruchsvolle Anwendungen.[McC24b; McC24a]

2.4.2. Wire

Die Wire Bibliothek ermöglicht die Kommunikation mit I2C-Geräten auf allen Arduino-Plattformen und ist ein weit verbreitetes Protokoll zum Lesen und Senden von Daten zu und von externen I2C-Komponenten. Die I2C-Pins variieren je nach Hardware-Design und Architektur der verschiedenen Arduino-Boards, wobei die Standard-I2C-Pins für Nano Boards die Pins A4 (SDA) und A5 (SDL) sind.

2.4.3. Adafruit GFX

Die Adafruit GFX Bibliothek ist eine Kern-Grafikbibliothek, die grundlegende Grafikprimitiven wie Punkte, Linien und Kreise für Adafruit-Displays bereitstellt. Sie ist unerlässlich für die Darstellung von Grafiken auf verschiedenen Hardware-Displays und erfordert für jedes Display eine spezifische Hardware-Bibliothek zur Verwaltung der niedrigeren Funktionen. Zusätzlich unterstützt sie verschiedene Bitmap-Schriftarten und bietet Werkzeuge wie Image2Code zur Konvertierung von BMP-Dateien in Code-Arrays für die Anzeige. Die Bibliothek legt besonderen Wert auf die Rückwärtskompatibilität mit bestehenden Arduino-Sketches und bietet umfassende Möglichkeiten zur Optimierung und Anpassung von Grafik- und Textdarstellungen auf den Displays. Sie wird in diesem Projekt in der Version 1.11.9 verwendet.[Aaaa]

2.4.4. Adafruit SSD1306

Die Adafruit SSD1306 Bibliothek ist speziell für monochrome OLED-Displays entwickelt, die den SSD1306 Treiber verwenden. Diese Displays kommunizieren über I2C oder SPI und erfordern 2 bis 5 Pins für die Verbindung. Die Bibliothek bietet grundlegende Funktionen zur Steuerung der Displays wie das Initialisieren der Kommunikation, das Zeichnen von Grafikelementen wie Linien und Kreisen sowie das Anzeigen von Text. Sie ermöglicht die einfache Anpassung der Displaygröße über den Konstruktor und unterstützt sowohl SPI-Transaktionen als auch die Spezifikation der SPI-Bitrate ab Arduino 1.6 oder höher. Die Installation erfolgt idealerweise über den Arduino IDE Bibliotheksmanager oder durch manuelles Herunterladen und Entpacken des Quellcodes von GitHub. Sie wird in diesem Projekt in der Version 2.5.10 verwendet.[Aaaa]

2.4.5. Encoder

Die Encoder Library ermöglicht das Zählen von Impulsen aus bestimmten Signalen, die häufig von Drehknöpfen, Motor- oder Wellensensoren sowie anderen Positionsgebern stammen. Diese Bibliothek ist für Arduino und Teensy Boards optimiert und bietet verschiedene Zählmodi, darunter 4X counting für präzise Positionserfassung. Die Verwendung erfolgt durch die Initialisierung eines Encoder-Objekts mit zwei Pins, wobei der erste Pin idealerweise über Interruptfähigkeit verfügt für optimale Leistung. Die Bibliothek unterstützt sowohl I2C- als auch SPI-Schnittstellen und bietet verschiedene Hardware- und Softwareoptionen zur Anpassung an die Anforderungen des Benutzers. Sie wird in diesem Projekt in der Version 1.4.4 verwendet.[Sto24]

2.5. Schrittmotorsteuerung

Für eine einfachere Bedienung des Schrittmotors wurde ein Schrittmotortreiber mit integriertem Übersetzer verbaut. Bei einer Leistung von bis zu 35 V und ± 2 A ist

der DEBO DRV A4988 für den Betrieb von bipolaren Schrittmotoren ausgelegt. Auf dem Treiber ist ein Stromregler mit fester Ausschaltzeit integriert, bei dem zwischen einem langsamen und einem gemischten Abschaltmodus gewählt werden kann. Durch eine Impulseingabe wird der Motor um einen Mikroschritt angetrieben. Der Vorteil der Schrittmotorsteuerung liegt darin, dass keine Phasensequenztabellen, Hochfrequenz-Steuerleitungen oder komplexe Schnittstellen programmiert werden müssen. Im Schrittbetrieb wählt der A4988 automatisch den Abklingmodus, langsam oder gemischt. Im gemischten Modus wird im Abklingvorgang zwischen dem schnellen und langsamen Modus gewechselt. Dabei führt der gemischte Modus zu einer Verringerung der hörbaren Motorgeräusche, einer höheren Schrittgenauigkeit und einer geringeren Verlustleistung. [All22; All21]

Abbildung 2.4.: Treiberkarte Debo DRV A4988

Weitere Details:

- **Abmessungen** ($b \times l \times h$): $5\text{ mm} \times 5\text{ mm} \times 0,9\text{ mm}$
- **Steuerspannung**: 3,3 - 5 V
- **Maximale Ausgangskapazität**: bis 35 V und $\mp 2\text{ A}$
- **Schutzschaltungen**: Thermische Abschaltschaltung, Schutz vor Masseschluss, Schutz vor kurzgeschlossener Last, Überkreuzungsstromschutz [All22]
- Das Board auf Basis des A4988 Treibers für Schrittmotoren bietet Auflösungen bis zu 1/16 Schritten.
- Die Auflösung kann manuell über einen DIP-Schalter gewählt werden.
- Richtungswahl
- einstellbare Stromstärke
- intelligente Chopping-Control
- Unterspannungs-, Übertemperaturschutz

2.6. Weiterführende Literatur

Die folgende Quelle ist ein technisches Tutorial von FAULHABER, das sich mit Schrittverlusten bei Schrittmotoren beschäftigt. Es erklärt, wie diese entstehen, wie man sie erkennt und welche Maßnahmen zu ihrer Vermeidung ergriffen werden können. Dabei werden typische Fehlerbilder (z. B. Stillstand, unvollständige Beschleunigung oder Synchronisationsverlust) analysiert und sowohl ursachenbezogene Diagnosemethoden als auch konkrete Lösungsansätze vorgestellt, etwa zur richtigen Motorauswahl, Anpassung von Betriebsparametern oder Optimierung der Ansteuerung. [Dr.20]

Teil III.

Domain Knowledge - Tools

Teil IV.

Development

Teil V.

Monitoring

Teil VI.

**Verification - Evaluation -
Conclusion**

Teil VII.

Anhang

Literaturverzeichnis

- [3dj] *Creality Endschalter*. 2026. URL: <https://www.3djake.de/creality-3d-drucker-ersatzteile/endschalter> (besucht am 26.04.2026).
- [Aaa] *Adafruit GFX Library*. [pdf]. 2024.
- [Aab] *Adafruit SSD1306 Library*. [pdf]. 2024.
- [All21] Allegro, Hrsg. *A4988-Datasheet - Short*. [pdf]. 2021. URL: www.allegromicro.com.
- [All22] Allegro, Hrsg. *A4988-Datasheet*. [pdf]. 2022. URL: <https://www.allegromicro.com>.
- [Arda] *Digital Pins - Arduino Reference*. 2024. URL: <https://docs.arduino.cc/language-reference/de/variablen/constants/inputOutputPullup/> (besucht am 25.04.2026).
- [Ardb] *Serial.print() - Arduino Reference*. 2024. URL: <https://www.arduino.cc/reference/de/language/functions/communication/serial/print/> (besucht am 25.04.2026).
- [Ardc] *delay() - Arduino Reference*. 2024. URL: <https://www.arduino.cc/reference/de/language/functions/time/delay/> (besucht am 25.04.2026).
- [Ardd] *digitalRead() - Arduino Reference*. 2024. URL: <https://www.arduino.cc/reference/de/language/functions/digital-io/digitalread/> (besucht am 25.04.2026).
- [Arde] *loop() - Arduino Reference*. URL: <https://www.arduino.cc/reference/de/language/structure/sketch/loop/> (besucht am 25.04.2026).
- [Ardf] *pinMode() - Arduino Reference*. 2024. URL: <https://www.arduino.cc/reference/de/language/functions/digital-io/pinMode/> (besucht am 25.04.2026).
- [Ardg] *setup() - Arduino Reference*. URL: <https://www.arduino.cc/reference/de/language/structure/sketch/setup/> (besucht am 25.04.2026).
- [Bab23] G. Babel. *Elektrische Antriebe in der Fahrzeugtechnik: Lehr- und Arbeitsbuch*. 5. Wiesbaden und Heidelberg: Springer Vieweg, 2023. ISBN: 978-3-658-40585-4. DOI: 10.1007/978-3-658-40586-1.
- [Bas] *Creality Endschalter*. 2026. URL: <https://www.bastelgarage.ch/creality-endschalter-limit-switch> (besucht am 26.04.2026).
- [Ber18] H. Bernstein. *Elektrotechnik/Elektronik für Maschinenbauer - Einfach und praxisgerecht*. 3. Aufl. Berlin Heidelberg New York: Springer-Verlag, 2018, S. 235–236. ISBN: 978-3-658-20838-7.
- [Ber24] H. Bernstein. *Messelektronik und Sensoren. Grundlagen der Messtechnik, Sensoren, analoge und digitale Signalverarbeitung*. 2. Aufl. Springer Vieweg Wiesbaden, 25. Jan. 2024. ISBN: 978-3-658-38929-1. DOI: <https://doi.org/10.1007/978-3-658-38929-1>.
- [Dr.20] Dr. Fritz Faulhaber GmbH & Co. KG, Hrsg. *Faulhaber Tutorial: Schrittverluste verhindern bei Schrittmotoren*. [pdf]. Schönaich Deutschland, 2020. URL: www.faulhaber.com/de/know-how/tutorials/schrittmotoren-tutorial-schrittverluste-verhindern-bei-schrittmotoren/.

-
- [Gloa] Global Electric Motor Solution LLC, Hrsg. *NEMA 17, 1.8° 42mm GM42BYG Stepper Motors - Zeichnung*. [\[pdf\]](#).
 - [Glob] Global Electric Motor Solution LLC, Hrsg. *NEMA 17, 1.8° 42mm GM42BYG Stepper Motors*. [\[pdf\]](#). URL: gemsmotor.com/stepper/nema17-stepper-motor.pdf.
 - [Hag21] R. Hagl. *Elektrische Antriebstechnik*. 3., überarbeitete und erweiterte Auflage. München: Hanser, 2021. ISBN: 978-3-446-46572-5. DOI: [10.3139/9783446468214](https://doi.org/10.3139/9783446468214).
 - [Hel] W. Helbig. *Praxiswissen in der Messtechnik. Arbeitsbuch für Techniker, Ingenieure und Studenten*.
 - [Her+12] E. Hering u. a. *Elektrotechnik und Elektronik für Maschinenbauer*. Bd. 2. Springer, 14. Feb. 2012. ISBN: 978-3-642-12881-3.
 - [McC24a] M. McCauley. *AccelStepper Library - Classes*. [\[pdf\]](#). 2024.
 - [McC24b] M. McCauley. *AccelStepper Library*. [\[pdf\]](#). 2024.
 - [McC24c] M. McComb. *Micro-stepping for Stepper Motors*. Hrsg. von Microchip Technology. [\[pdf\]](#). 2024.
 - [SK21] D. Schröder und R. Kennel. *Elektrische Antriebe – Grundlagen*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2021. ISBN: 978-3-662-63100-3. DOI: [10.1007/978-3-662-63101-0](https://doi.org/10.1007/978-3-662-63101-0).
 - [Sto24] P. Stoffregen. *Encoder Library*. Hrsg. von PJRC. [\[pdf\]](#). 2024. URL: https://www.pjrc.com/teensy/td_libs_Encoder.html.
 - [TR14] H.-R. Tränkler und L. Reindl, Hrsg. *Sensortechnik: Handbuch für Praxis und Wissenschaft*. 2. Aufl. VDI-Buch. Published: 13 January 2015. eBook Packages: Computer Science and Engineering (German Language). Berlin, Heidelberg: Springer Vieweg Berlin, Heidelberg, 2014, S. XIV, 1596. ISBN: 978-3-642-29942-1. DOI: [10.1007/978-3-642-29942-1](https://doi.org/10.1007/978-3-642-29942-1).
 - [Zie25] D. Zielke. *Mikrosysteme*. Bd. 1. Springer, 5. März 2025. ISBN: 978-3-662-71232-0.